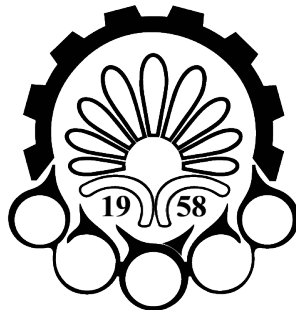


به نام خدا



Amirkabir University of Technology
(Tehran Polytechnic)

دانشکده مهندسی
کامپیوتر و فناوری اطلاعات

مستندات پروژه درس مهندسی نرم افزار ۲

کاوه احمدی ۴۰۱۳۱۹۰۴

آرمینا متقی ۴۰۱۳۱۴۱۸

تیر ۱۴۰۵

۱ Product Vision

پلتفرم End-to-End Ticketing یک سیستم با High Availability برای مدیریت Flash Sales و ترافیک بالا است. هدف سیستم جلوگیری از Downtime و مقیاس پذیری پویا برای توسعه سیستم است.

۱.۱ کاربران هدف

- End-User نیازمند جستجوی سریع رویداد، مشاهده نقشه Real-Time سالن، قرارگیری در Virtual Waiting Room و دریافت QR Code.
- Admin نیازمند طراحی سالن چندبخشی، تنظیم Dynamic Pricing و مشاهده Dashboards تحلیلی در لحظه.

۲.۱ معیارهای کلیدی عملکرد

- آ) Checkout Completion Rate بررسی درصد موفقیت پرداخت پس از قفل صندلی.
- ب) Seat Lock Expiration Rate بررسی نرخ انقضای قفل های موقت Redis.
- ج) Latency حفظ زمان پاسخگویی زیر یک ثانیه و پایداری Uptime بالا.
- د) Gross Merchandise Value پیگیری درآمدزایی سیستم به صورت زنده.

۲ Risk Mitigation Analysis

۱.۲ ریسک اول Flash Traffic Spikes

شرح ریسک: هجوم همزمان ربات ها و کاربران به API Gateway و خطر از کار افتادن Event Catalog.
راهکار مهندسی:

- اجرای Virtual Waiting Room برای Traffic Throttling.
- تنظیم Rate Limiting در لایه Edge.
- استفاده از Auto-scaling در Kubernetes.

۲.۲ ریسک دوم Race Conditions

شرح ریسک: درخواست همزمان برای یک صندلی و خطر Double-Booking.
راهکار مهندسی:

- پیاده سازی Distributed Locks توسط Redis با Time-To-Live مشخص.
- استفاده از Optimistic Concurrency Control در PostgreSQL.

۳.۲ ریسک سوم Payment Gateway Timeout

شرح ریسک: قطعی درگاه بانکی در حین Checkout و گیر کردن صندلی در حالت رزرو. راهکار مهندسی:

- اجرای Saga Pattern و تراکنش‌های Compensation.
- آزادسازی خودکار قفل‌ها و بازگشت صندلی به سیستم در صورت دریافت خطا.

۴.۲ ریسک چهارم Network Partitioning

شرح ریسک: قطعی ارتباط بین سرویس‌ها مانند عدم ارسال SMS تایید به مشتری. راهکار مهندسی:

- استفاده از Asynchronous Messaging توسط Message Brokers.
- ارسال پیام‌های ناموفق به Dead-Letter Queue جهت تلاش مجدد.

۳ Diagrams

۱.۳ Use Case Diagram

این بخش شامل Use Case Diagram سیستم است که تعامل بین Actors و Use Cases را نشان می‌دهد.

۱.۱.۳ تحلیل اکترها

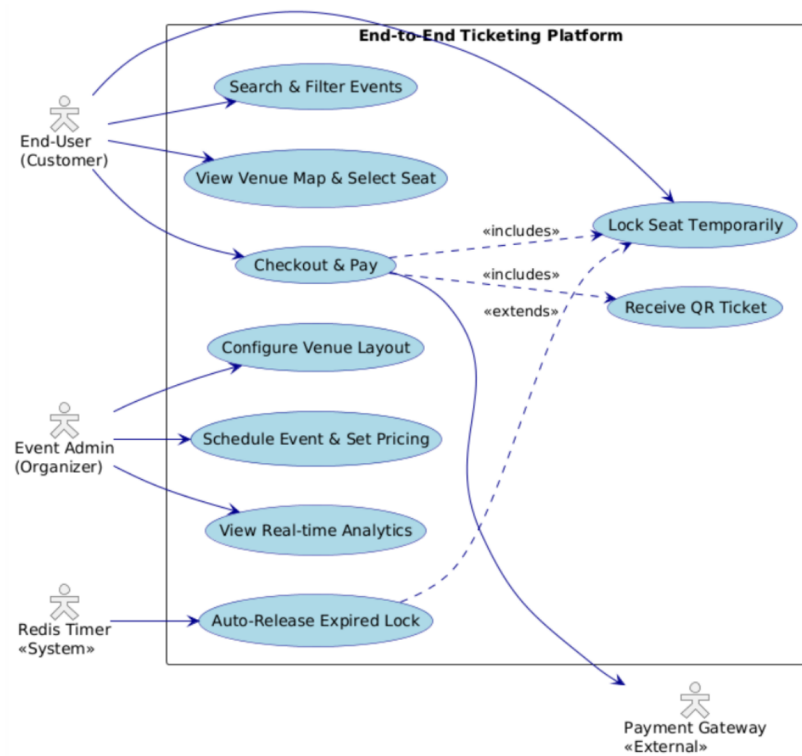
- End-User خریدار بلیت که عملیاتی مانند جستجو انتخاب صندلی و Checkout را انجام می‌دهد.
- Event Admin مدیر سیستم که وظیفه Configure Venue Layout و بررسی Analytics را بر عهده دارد.
- Payment Gateway یک External Actor برای پردازش تراکنش‌های مالی.
- Redis Timer یک System Actor برای مدیریت زمان انقضای قفل صندلی‌ها.

۲.۱.۳ تحلیل روابط

- Includes کاربر برای تکمیل Checkout حتما باید مرحله Lock Seat Temporarily را انجام دهد.
- Extends در صورت عدم پرداخت در زمان مقرر سیستم به صورت خودکار Auto-Release Expired Lock را اجرا می‌کند تا صندلی آزاد شود.

۲.۳ Sequence Diagram

این بخش Sequence Diagram فرآیند رزرو و پرداخت بلیت را نشان می‌دهد که تعامل بین کامپوننت‌ها را در طول زمان مشخص می‌کند.

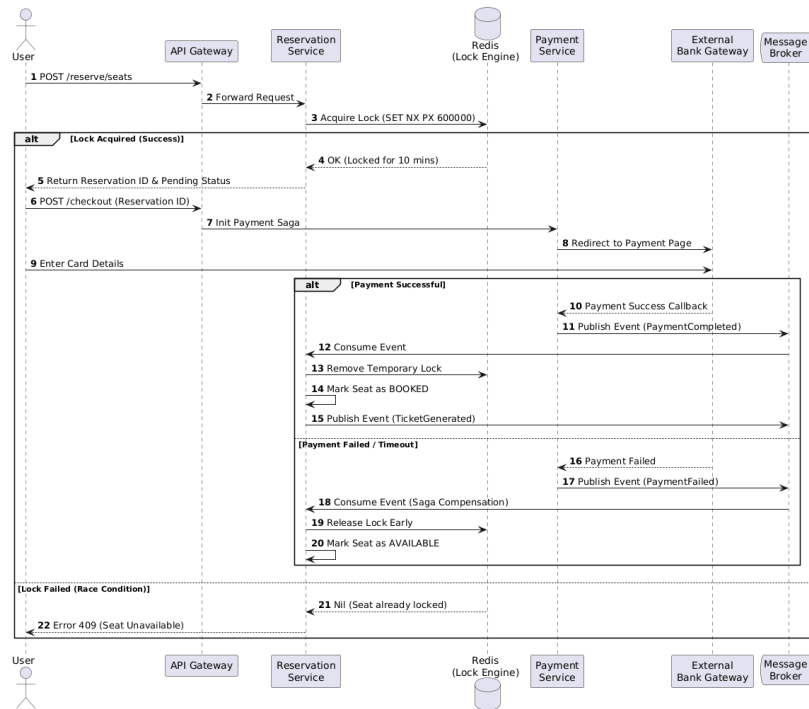


۱.۲.۳ تحلیل جریان اصلی

- Lock Acquisition کاربر درخواست POST /reserve/seats را ارسال می کند و Reservation Service با دستور SET NX PX 600000 صندلی را در Redis برای ده دقیقه قفل می کند.
- Payment Saga پس از دریافت Reservation ID کاربر عملیات Checkout را آغاز کرده و به صفحه پرداخت هدایت می شود.

۲.۲.۳ تحلیل سناریوها

- Payment Successful پس از موفق Callback Payment Service رویداد PaymentCompleted را منتشر می کند سپس قفل موقت حذف شده و صندلی به وضعیت BOOKED تغییر می کند و رویداد TicketGenerated منتشر می شود.
- Payment Failed در صورت شکست یا Timeout رویداد PaymentFailed منتشر شده و Saga Compensation اجرا می شود تا قفل زودتر آزاد و صندلی به وضعیت AVAILABLE برگردد.
- Race Condition اگر صندلی از قبل قفل شده باشد Redis مقدار Nil برمی گرداند و سیستم خطای Error 409 را نمایش می دهد.



Deployment Diagram ۳.۳

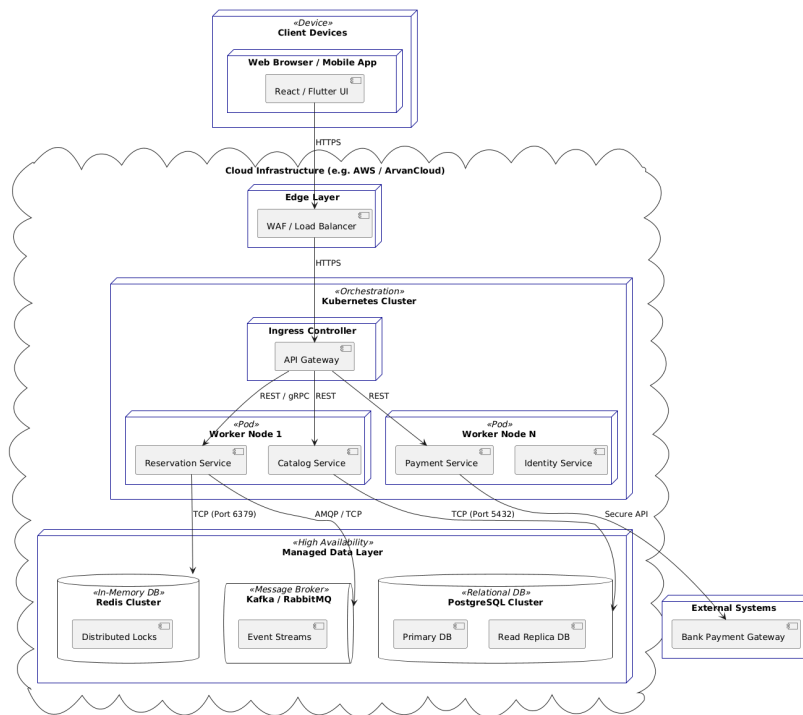
این بخش Deployment Diagram است که نحوه استقرار کامپوننت‌ها بر روی زیرساخت فیزیکی و ابری را نمایش می‌دهد.

۱.۳.۳ تحلیل لایه‌ها

- Client Devices شامل Web Browser و Mobile App است که با استفاده از React / Flutter UI پیاده‌سازی شده و از طریق HTTPS ارتباط برقرار می‌کند.
- Edge Layer شامل WAF / Load Balancer برای مدیریت ترافیک ورودی و امنیت است.
- Kubernetes Cluster یک لایه Orchestration که شامل Ingress Controller و Worker Node های مختلف برای اجرای سرویس‌ها است.
- Managed Data Layer با قابلیت High Availability شامل Redis Cluster و Kafka / RabbitMQ و PostgreSQL Cluster است.

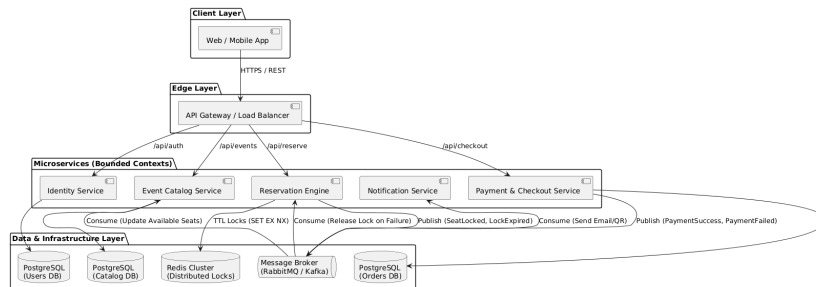
۲.۳.۳ تحلیل ارتباطات

- Redis Connection سرویس‌ها از طریق TCP Port 6379 به Distributed Locks دسترسی دارند.
- Database Connection ارتباط با PostgreSQL از طریق TCP Port 5432 انجام می‌شود.
- External API پرداخت‌ها از طریق Secure API به Bank Payment Gateway متصل می‌شوند.



۴.۳ Component Diagram

این بخش Component Diagram است که کامپوننت‌های اصلی سیستم و ارتباط بین Microservices را در قالب Bounded Contexts نشان می‌دهد.



۱.۴.۳ تحلیل لایه‌ها

- Client Layer شامل Web / Mobile App است که از طریق HTTPS / REST با سیستم ارتباط برقرار می‌کند.
- Edge Layer شامل API Gateway / Load Balancer است که درخواست‌ها را به سرویس‌های مربوطه هدایت می‌کند.

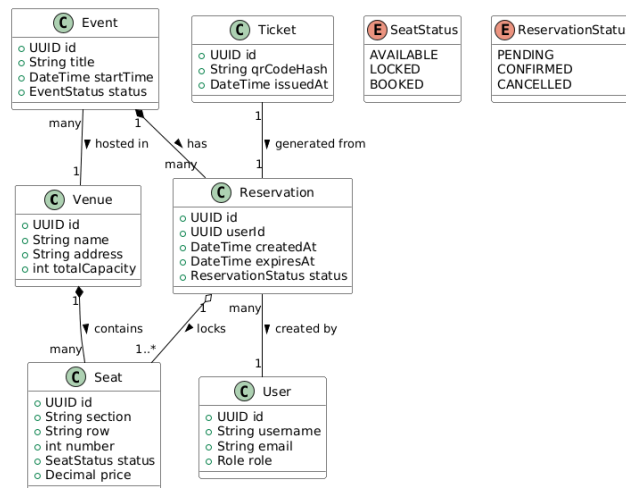
- Microservices شامل Identity Service و Event Catalog Service و Reservation Engine و Notification Service و Payment & Checkout Service است.
- Data & Infrastructure Layer شامل چند PostgreSQL مجزا و Redis Cluster و Message Broker است.

۲.۴.۳ تحلیل تعاملات رویدادمحور

- Distributed Locks موتور رزرو از TTL Locks با دستور SET EX NX در Redis استفاده می کند.
- Event Publishing رویدادهای SeatLocked و LockExpired و PaymentSuccess و PaymentFailed از طریق Message Broker منتشر می شوند.
- Event Consuming سرویس ها رویدادها را برای Update Available Seats و Release Lock on و Send Email/QR و Failure مصرف می کنند.

۵.۳ Class Diagram

این بخش Class Diagram است که ساختار کلاس ها و روابط بین موجودیت های اصلی سیستم را نمایش می دهد.



۱.۵.۳ تحلیل کلاس ها

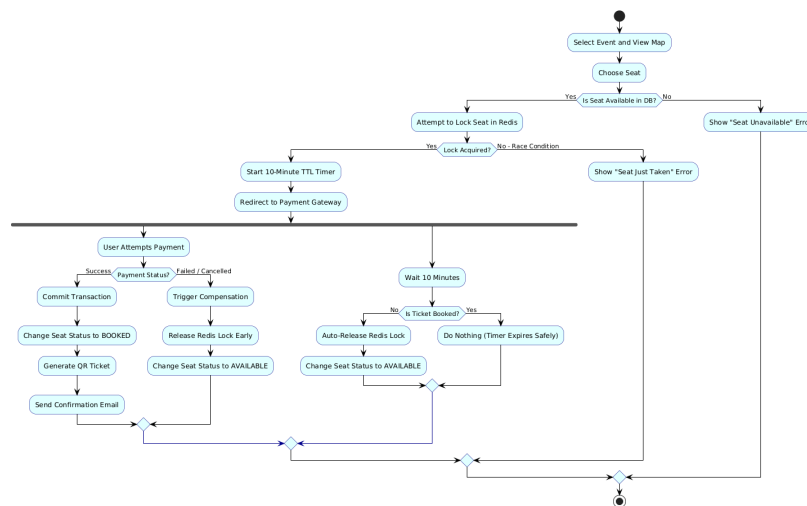
- Event رویداد اصلی که شامل title و startTime و status است و در یک Venue برگزار می شود.
- Venue محل برگزاری که شامل name و address و totalCapacity است و شامل چندین Seat می باشد.
- Reservation رزرو که با userId و createdAt و expiresAt مشخص می شود و صندلی ها را قفل می کند.
- Ticket بلیت نهایی که شامل qrCodeHash است و از یک Reservation تولید می شود.
- Seat صندلی که شامل section و row و number و price است.
- User کاربر که شامل username و email و role است.

۲.۵.۳ Enumerations تحلیل

- SeatStatus شامل مقادیر AVAILABLE و LOCKED و BOOKED است.
- ReservationStatus شامل مقادیر PENDING و CONFIRMED و CANCELLED است.

۶.۳ Activity Diagram

این بخش Activity Diagram است که جریان کاری فرآیند رزرو صندلی و پرداخت را همراه با شاخه‌های تصمیم‌گیری نشان می‌دهد.



۱.۶.۳ تحلیل جریان انتخاب صندلی

- Seat Availability پس از Choose Seat سیستم بررسی می‌کند که آیا صندلی در پایگاه داده موجود است در غیر این صورت خطای Seat Unavailable نمایش داده می‌شود.
- Lock Attempt در صورت موجود بودن سیستم تلاش می‌کند صندلی را در Redis قفل کند و اگر Race Condition رخ دهد خطای Seat Just Taken نمایش داده می‌شود.
- TTL Timer پس از قفل موفق یک تایمر ده دقیقه‌ای شروع شده و کاربر به Payment Gateway هدایت می‌شود.

۲.۶.۳ تحلیل جریان پرداخت

- Payment Success در صورت موفقیت تراکنش Commit شده و صندلی به BOOKED تغییر می‌کند سپس QR Ticket تولید و ایمیل تأیید ارسال می‌شود.
- Payment Failed در صورت شکست یا لغو Compensation فعال شده و قفل زودتر آزاد و صندلی به AVAILABLE برمی‌گردد.
- Timeout Handling در صورت گذشت ده دقیقه اگر بلیت Booked نشده باشد قفل Redis به صورت خودکار آزاد می‌شود در غیر این صورت تایمر بدون تأثیر منقضی می‌شود.

۴ زیرساخت به عنوان کد Infrastructure as Code

در این بخش معماری زیرساخت ابری پلتفرم با استفاده از اسکریپت‌های Terraform مستند شده است. هدف از این رویکرد اتوماسیون استقرار مقیاس‌پذیری سریع در زمان ترافیک بالا و جلوگیری از خطاهای انسانی در پیکربندی سرورها است.

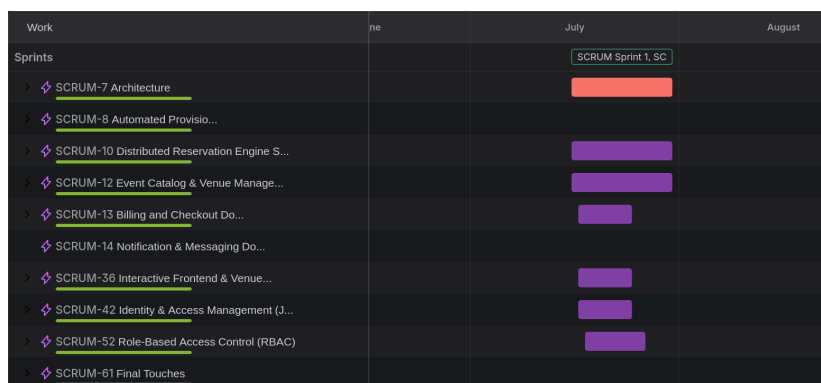
۱.۴ تحلیل منابع ابری تخصیص یافته

فایل پیکربندی main.tf وظیفه راه‌اندازی خودکار منابع زیر را بر عهده دارد

- Virtual Private Cloud ایجاد یک شبکه مجازی ایزوله برای تامین امنیت شبکه و محدود کردن دسترسی مستقیم به پایگاه‌های داده.
- Application Load Balancer استقرار مسیریاب ترافیک در لایه لبه Edge Layer برای توزیع متوازن درخواست‌های کاربران بین نودهای پردازشی و جلوگیری از افت کیفیت سرویس.
- Managed Kubernetes Cluster پیکربندی کلاستر EKS جهت مدیریت کانتینرها میکروسرویس‌ها و اعمال مقیاس‌پذیری خودکار Auto-scaling در زمان هجوم ترافیک یا همان Flash Sales.
- Managed PostgreSQL راه‌اندازی پایگاه داده رابطه‌ای با قابلیت Multi-AZ برای تضمین پایداری داده‌های مالی کاتالوگ رویدادها و دسترسی‌پذیری بالا High Availability.
- Redis Cluster تخصیص کلاستر در حافظه In-Memory برای مدیریت سریع قفل‌های توزیع شده Dis-tributed Locks و جلوگیری از خطای همزمانی درخواست‌ها Race Conditions.
- Message Broker استقرار RabbitMQ برای مدیریت صف‌های پیام و تضمین ارتباطات غیرهمزمان Asynchronous بین سرویس‌های رزرواسیون پرداخت و اطلاع‌رسانی.

۵ مدیریت چرخه حیات چابک و تضمین کیفیت

توسعه پلتفرم با رویکرد تکرارپذیر و بر اساس متدولوژی Scrum مدیریت می‌شود. برای پیگیری وظایف و کنترل کیفیت کدها از ابزارهای Jira و GitLab CI/CD استفاده شده است.



شکل ۱: نمای Timeline پروژه و توزیع وظایف در Jira

۱.۵ ساختار شکست کار و عملیات چابک

نیازهای سیستم به دامنه‌های سطح بالا تحت عنوان اپیک Epic تقسیم شده‌اند. یکی از حیاتی‌ترین اپیک‌های این سیستم Distributed Reservation Engine Setup است. برای درک بهتر نیازمندی‌ها از داستان‌های کاربر User Stories استفاده شده است. به عنوان مثال برای مدیریت قفل صندلی داستان زیر تعریف شده است
به عنوان یک مشتری من می‌خواهم صندلی انتخاب شده‌ام به مدت ده دقیقه قفل شود تا بتوانم بدون استرس از دست دادن آن فرایند پرداخت را تکمیل کنم.

۲.۵ استراتژی تضمین کیفیت و تست یکپارچگی

با توجه به ماهیت ترافیک بالا در پلتفرم‌های فروش بلیت و خطر بروز خطای فروش مضاعف Overbooking استراتژی‌های تست زیر در نظر گرفته شده‌اند.

- Load and Stress Testing نوشتن اسکریپت‌های تست برای شبیه‌سازی هجوم هزاران درخواست همزمان از سوی ربات‌ها Bot Requests به یک صندلی خاص جهت سنجش پایداری قفل‌های Redis.
- Mutation Testing به عنوان یک استراتژی پیشرفته پیشنهاد می‌شود که با ایجاد تغییرات عمدی در کد هسته رزرواسیون، از توانایی تست‌های Unit Tests در کشف خطاهای منطقی اطمینان حاصل می‌کند و پوشش کد Code Coverage را در ماژول‌های تراکنشی می‌سنجد.
- Automated Verifications اجرای خودکار تست‌ها در هر Commit با استفاده از GitLab CI برای جلوگیری از ورود کدهای معیوب به محیط عملیاتی.

۶ سرویس‌های توزیع شده استقرار عملیاتی و مدیریت حوادث

این بخش به بررسی استراتژی‌های پیاده‌سازی سیستم در محیط عملیاتی جداسازی دامنه‌ها و نحوه برخورد تیم مهندسی با حوادث غیرمترقبه می‌پردازد.

۱.۶ مرزبندی سرویس‌ها بر اساس طراحی مبتنی بر دامنه

تیم مهندسی با استفاده از اصول طراحی مبتنی بر دامنه Domain-Driven Design مرزهای سیستم Bounded Contexts را به گونه‌ای تعریف کرده است که وابستگی ساختاری و چرخشی میان سرویس‌ها از بین برود. به عنوان مثال بررسی وضعیت در دسترس بودن صندلی‌ها از دامنه کاتالوگ رویدادها کاملاً جدا شده و منحصراً در کنترل موتور رزرواسیون قرار گرفته است. این معماری به تیم‌ها اجازه می‌دهد تا سرویس‌های پرفشار را به صورت مستقل مقیاس‌دهی کنند.

۲.۶ نظارت عملیاتی و ارتباطات غیرهمزمان

برای جلوگیری از قطعی سیستم در زمان ترافیک لحظه‌ای از معماری پیام‌محور Asynchronous Messaging استفاده شده است. نتایج تراکنش‌ها از طریق صف‌های پیام به سرویس اطلاع‌رسانی منتقل می‌شوند تا بار پردازشی کاهش یابد.

همچنین در طراحی معماری، استقرار سیستم بر روی کلاستر Kubernetes پیشنهاد شده است تا از مقیاس‌پذیری افقی خودکار Horizontal Auto-scaling و استقرار بدون قطعی Zero-downtime Canary Upgrades پشتیبانی شود. برای پایش وضعیت سلامت سیستم، استفاده از Prometheus و Grafana به عنوان راهکار پیشنهادی در نظر گرفته شده است.

۳.۶ گزارش کالبدشکافی قطعی‌ها Incident Postmortems

برای مدیریت حوادث چرخش‌های شیفت پاسخگویی On-Call Rotations تعریف شده است تا در صورت عبور متریک‌ها از آستانه خطر تیم مهندسی بلافاصله وارد عمل شود. در ادامه سناریوهای قطعی بحرانی تحلیل شده‌اند

- **Payment Processing Outages** در این سناریو درگاه پرداخت خارجی دچار قطعی و پایان زمان انتظار Timeout می‌شود. در نتیجه صندلی‌های رزرو شده در حالت قفل معلق می‌مانند. راهکار مهندسی اجرای اسکریپت‌های جبرانی Compensation Scripts است که با دریافت خطای پرداخت از طریق صف پیام بلافاصله قفل سیستم Redis را می‌شکنند و صندلی‌ها را به چرخه فروش بازمی‌گردانند.
- **Queue Worker Failures** در این سناریو سیستم اتاق انتظار مجازی تحت فشار هجوم میلیونی کاربران دچار فروپاشی می‌شود. سیستم به گونه‌ای طراحی شده است که در صورت از کار افتادن این ماژول درخواست‌های جدید بدون درگیر کردن هسته مرکزی دیتابیس با خطای استاندارد عدم سرویس‌دهی موقت HTTP 503 Service Unavailable مواجه شوند تا از فروپاشی کل سیستم جلوگیری گردد.
- **Inter-Service Network Partitioning** در این سناریو ارتباط شبکه‌ای بین درگاه ورودی API Gateway و موتورهای داخلی قطع می‌شود. برای رفع این مشکل استفاده از الگوی قطع‌کننده مدار Circuit Breaker پیشنهاد می‌شود تا ضمن جلوگیری از انباشت درخواست‌های ناموفق سیستم به صورت خودکار پس از بازیابی شبکه عملیات را از سر بگیرد.

۷ طراحی و تقسیم‌بندی میکروسرویس‌ها

سیستم فروش بلیت ما بر اساس اصول Domain-Driven Design به پنج دامنه‌ی مستقل تقسیم شده است. هر دامنه یک مسئولیت مشخص دارد و از طریق REST API یا پیام‌رسان RabbitMQ با سایر دامنه‌ها ارتباط برقرار می‌کند. این تقسیم‌بندی باعث کاهش وابستگی و امکان independent scaling می‌شود.

سرویس هویت و دسترسی مسئول login, registration و صدور JWT token است. کنترل دسترسی مبتنی بر نقش در این سرویس پیاده‌سازی شده و کاربران را به دو گروه Admin و Customer تقسیم می‌کند. برای امنیت بیشتر، token در قالب httpOnly Cookie ذخیره می‌شود تا در برابر حملات XSS مقاوم باشد.

سرویس کاتالوگ و رویداد اطلاعات سالن‌ها، رویدادها و صندلی‌ها را نگهداری می‌کند. ساختار هر صندلی شامل row, section و seat number است و وضعیت آن در یکی از سه حالت Available, Locked یا Booked قرار دارد.

سرویس رزرو و موجودی هسته‌ی اصلی سیستم است و مسئولیت concurrency control را بر عهده دارد. برای جلوگیری از double-booking، از دستور اتمیک SET NX در Redis استفاده می‌شود. هر قفل دارای یک TTL است تا در صورت رها شدن، صندلی به‌طور خودکار آزاد شود.

سرویس پرداخت و تسویه درخواست پرداخت را دریافت کرده و پس از تأیید مالکیت قفل، یک پیام موفقیت را به صف RabbitMQ ارسال می‌کند. جداسازی این سرویس از عملیات ذخیره‌سازی، باعث fault tolerance و پاسخ‌دهی سریع به کاربر می‌شود.

سرویس پردازش و اعلان به‌صورت یک worker مستقل عمل می‌کند و پیام‌های پرداخت را از صف دریافت می‌کند. سپس وضعیت صندلی را در پایگاه‌داده به Booked تغییر داده و قفل مربوطه را آزاد می‌کند. این معماری asynchronous تضمین می‌کند که حتی در صورت قطعی موقت پایگاه‌داده، هیچ پیامی از دست نرود.